

Chapter 9 Priority Queues

The Priority Queue ADT (Abstract Data Type)

Prompt

What is a priority queue?

Answer

A **priority queue** is a collection of items where:

- Each item has a **key** (priority)
- Each item has a **value** (data)
- The item with the **minimum key** always has the **highest priority**

⚠ Important:

- This is **not** a FIFO queue
- Removal order depends on **priority**, not insertion time

Prompt

How is an element represented in a priority queue?

Answer

Each element is stored as a **(key, value)** pair:

- **Key (k)**: determines priority (smaller = higher priority)
- **Value (v)**: associated data

Example:

`(3, "B")` → `priority = 3, value = "B"`

Prompt

What operations must a priority queue support?

Answer

A priority queue P supports:

Operation	Description
<code>P.add(k, v)</code>	Insert an item with key k and value v

<code>P.min()</code>	Return (k, v) with minimum key , without removing
<code>P.remove_min()</code>	Remove and return (k, v) with minimum key
<code>P.is_empty()</code>	Return <code>True</code> if empty
<code>len(P)</code>	Return number of items

Prompt

What happens if keys are equal?

Answer

- Multiple items **may have the same key**
- `min()` and `remove_min()` may return **any one** of them
- No ordering guarantee between equal-priority elements

Prompt

Can keys change after insertion?

Answer

- **No**, in the basic ADT
- Keys are **fixed once inserted**
- Key updates are a **separate extension** (covered later)

Let's trace the given example carefully ☞

◆ Start

`P = {}`

◆ Operation: `P.add(5, A)`

`P = {(5, A)}`

◆ Operation: `P.add(9, C)`

`P = {(5, A), (9, C)}`

◆ Operation: `P.add(3, B)`

`P = {(3, B), (5, A), (9, C)}`

◆ Operation: `P.add(7, D)`

`P = {(3, B), (5, A), (7, D), (9, C)}`

⚠ This sorted display is conceptual only
The priority queue **does not** have to store items sorted

◆ **Operation:** `P.min()`

Prompt

What does `min()` return?

Answer

- Returns the item with **smallest key**
- Does **not** remove it

Return → (3, B)

P unchanged

◆ **Operation:** `P.remove_min()`

Return → (3, B)

P = {(5, A), (7, D), (9, C)}

◆ **Operation:** `P.remove_min()`

Return → (5, A)

P = {(7, D), (9, C)}

◆ **Operation:** `len(P)`

Return → 2

◆ **Operation:** `P.remove_min()`

Return → (7, D)

P = {(9, C)}

◆ **Operation:** `P.remove_min()`

Return → (9, C)

P = {}

◆ **Operation:** `P.is_empty()`

Return → True

◆ Operation: `P.remove_min()`

Error (queue is empty)

Prompt

How is a priority queue different from a normal queue?

Answer

Feature	Queue	Priority Queue
Removal order	FIFO	Minimum key
Priority-based	✗	✓
Uses keys	✗	✓
Real-world use	Lines	Scheduling, graphs

Prompt

Why are priority queues important?

Answer

They are used in:

- CPU scheduling
- Dijkstra's shortest path algorithm
- Prim's minimum spanning tree
- Event-driven simulations
- Task management systems

The Composition Design Pattern (for Priority Queues)

Prompt

What challenge arises when implementing a priority queue?

Answer

In a priority queue:

- Each entry has **two parts**:
 - a **key** (priority)
 - a **value** (data)
- During operations like insertion, deletion, or heap reordering, items **move around**

✱ Problem:

If keys and values are stored separately, they can easily get **out of sync** when items move.

Prompt

What is the composition design pattern?

Answer

The **composition design pattern** bundles related data into a **single object**, ensuring they always move together.

✂ Instead of:

```
key list      value list
```

We use:

```
Item(key, value)
```

So when an item moves, **both key and value move together safely**.

Prompt

Why is composition especially useful for priority queues?

Answer

Because priority queues:

- Compare items by **key**
- Store arbitrary **values**
- Frequently **relocate items** internally (e.g., heap swaps)

Composition guarantees:

- ✓ Key and value stay paired
- ✓ Cleaner code
- ✓ Fewer bugs
- ✓ Easy comparison logic

PriorityQueue Base Design

To standardize all priority-queue implementations, we define:

```
PriorityQueueBase
└─ Item (nested class)
```

Prompt

What is the purpose of the nested `Item` class?

Answer

The `Item` class:

- Stores **key** and **value together**
- Defines how items are **compared**
- Acts as the **atomic unit** inside the priority queue

✦ Code Structure (Conceptual)

```
class PriorityQueueBase:
    class Item:
        def __init__(self, k, v):
            self.key = k
```

```
self.value = v
```

Each element in the queue is now:

```
Item(key, value)
```

Prompt

How do priority queue items compare to each other?

Answer

Items are compared **only by their keys**, not values.

```
def __lt__(self, other):  
    return self.key < other.key
```

This means:

- $(3, "B") < (5, "A") \rightarrow \checkmark$
- Values don't affect priority

🔗 Why this matters

This allows:

- Sorting
- Heap ordering
- Efficient min extraction

All without knowing or caring what the value is.

Prompt

Why use `__slots__` in the Item class?

Answer

```
__slots__ = '_key', '_value'
```

This:

- Saves memory
- Prevents dynamic attribute creation

- Makes items **lighter and faster**

Perfect for large priority queues.

Prompt

Why is `is_empty()` implemented in the base class?

Answer

```
def is_empty(self):
    return len(self) == 0
```

Because:

- All priority queues must support `len()`
- `is_empty()` logic is **identical** for all implementations

- ✓ Code reuse
- ✓ Less duplication
- ✓ Cleaner subclasses

🧐 Without Composition (BAD)

Keys: [3, 5, 7]

Values: ["B", "A", "D"]

Swap keys → values may mismatch ✗

✓ With Composition (GOOD)

Priority Queue Storage

```
[
| Item(3, "B") |
| Item(5, "A") |
| Item(7, "D") |
]
```

Swapping items keeps **key + value together** ✓

↻ Movement Example (Heap Swap)

Before swap:

Item(5, "A") ↔ Item(3, "B")

After swap:

Item(3, "B") ↔ Item(5, "A")

No risk of mismatch ☞

10 Big Picture Architecture

PriorityQueueBase

```
|
|— Item (key, value)
|
|— is_empty()
|
|— Abstract methods:
|   |— add()
|   |— min()
|   |— remove_min()
|   |— __len__()
```

All concrete priority queues reuse this design.

Priority Queue Implementation Using an Unsorted List

Prompt

What is the main idea behind implementing a priority queue with an unsorted list?

Answer

We store all priority queue items in an **unsorted doubly linked list**.

- Insertion is **fast** $\rightarrow O(1)$
- Finding/removing the minimum is **slow** $\rightarrow O(n)$

This implementation favors **fast inserts** over fast deletes.

Prompt

What data structure is used internally?

Answer

A **PositionalList** implemented as a **doubly linked list**.

Each node stores:

```
Item(key, value)
```

✦ Reason:

- Constant-time insertions and deletions
- Flexible node positions

Prompt

How are key-value pairs stored?

Answer

Each entry is stored as an **Item composite**:

```
Item(k, v)
```

This comes from `PriorityQueueBase`.

Prompt

What happens when a new `UnsortedPriorityQueue` is created?

Answer

```
self.data = PositionalList()
```

- The list starts empty
- Number of items = list length

Prompt

How is the size of the priority queue determined?

Answer

```
def __len__(self):  
    return len(self.data)
```

- Directly returns list length
- Runs in **O(1)** time

Prompt

How is `is_empty()` implemented?

Answer

Inherited from `PriorityQueueBase`:

```
return len(self) == 0
```

- Uses `len()`
- Runs in **O(1)** time

Prompt

How does insertion work in an unsorted priority queue?

Answer

```
self.data.add_last(Item(key, value))
```

- Item added to **end of list**
- No sorting performed
- Takes **O(1)** time

Diagram: Insertion

Before `add(3, B)`:

`[ (5,A) ] → [ (9,C) ]`

After `add(3, B)`:

`[ (5,A) ] → [ (9,C) ] → [ (3,B) ]`

Prompt

Why is `min()` expensive in this implementation?

Answer

Because the list is **unsorted**, we must:

- Scan **every element**
- Compare keys
- Track the smallest one

This takes **$O(n)$** time.

Prompt

What does `find_min()` do?

Answer

It:

1. Starts at the first position
2. Walks through the entire list
3. Keeps track of the smallest key
4. Returns the **position** of the minimum item

Core Logic (Explained)

```
small = first position
```

```
walk = position after small
```

```
while walk exists:
```

```
    if walk.key < small.key:
```

```
        small = walk
```

walk = next position

Diagram: Finding Minimum

Start:

small $\rightarrow$ (5,A)

Compare with (9,C) $\rightarrow$ no change

Compare with (3,B) $\rightarrow$ update small

Final minimum $\rightarrow$ (3,B)

Prompt

How does the `min()` method work?

Answer

1. Calls `find_min()`
2. Retrieves the item at that position
3. Returns (key, value)
4. Does **not remove** the item

 Time complexity: **$O(n)$**

Prompt

How does `remove_min()` work?

Answer

1. Calls `find_min()`
2. Deletes that position from the list
3. Returns (key, value)

 Time complexity:

- Find min $\rightarrow O(n)$
- Delete $\rightarrow O(1)$
- Total $\rightarrow O(n)$

Diagram: Removal

Before:

[(5,A)] → [(3,B)] → [(7,D)]

Remove min:

delete (3,B)

After:

[(5,A)] → [(7,D)]

Prompt

Why does `find_min()` return a position instead of the item itself?

Answer

Because:

- `remove_min()` needs the **position** to delete in $O(1)$
- `PositionalList` deletes by position, not by value

Smart design 👉

Time Complexity Summary

Operation	Time
<code>len()</code>	$O(1)$
<code>is_empty()</code>	$O(1)$
<code>add(k, v)</code>	$O(1)$
<code>min()</code>	$O(n)$
<code>remove_min()</code>	$O(n)$

 Space complexity: **$O(n)$**

Prompt

When should we use an unsorted priority queue?

Answer

Best when:

- Many insertions
- Few removals
- Small dataset
- Simplicity matters more than speed

✗ Not ideal when frequent `remove_min()` calls are required.

High-Level Architecture Diagram

UnsortedPriorityQueue

|

├─ PositionalList (doubly linked list)

|

├─ Item(key, value)

|

├─ add() → $O(1)$

├─ find_min() → $O(n)$

├─ min() → $O(n)$

└─ remove_min() → $O(n)$

Priority Queue Implementation Using a Sorted List

Prompt

What is the main idea behind implementing a priority queue with a sorted list?

Answer

All items are stored in a **positional list sorted by nondecreasing keys**.

☞ This guarantees:

- **Minimum element is always at the front**
- Fast `min()` and `remove_min()`

📉 Trade-off:

- Insertions become **slow**

Prompt

What data structure is used internally?

Answer

- A **PositionalList**
- Implemented using a **doubly linked list**
- Each node stores an **Item(key, value)**

Prompt

What invariant is maintained at all times?

Answer

The list is always sorted in **nondecreasing order of keys**:

$k_1 \leq k_2 \leq k_3 \leq \dots \leq k_n$

★ This invariant is what enables $O(1)$ minimum access.

Prompt

What happens when a SortedPriorityQueue is created?

Answer

```
self.data = PositionalList()
```

- Empty list
- Size = 0

Prompt

How are size and emptiness checked?

Answer

- `len()` → returns length of positional list → **O(1)**
- `is_empty()` → inherited from base class → **O(1)**

Prompt

How does insertion work in a sorted priority queue?

Answer

To maintain sorted order:

1. Create a new `Item(k, v)`
2. Start from the **last element**
3. Walk backward until the correct position is found
4. Insert the item

Logic Explained

```
walk = last position
```

```
while walk exists AND newest.key < walk.key:
```

```
    walk = previous position
```

```
if walk is None:
```

```
    insert at front
```

```
else:
```

```
    insert after walk
```

Diagram: Insertion Example

Insert (4, X) into:

[(1,A)] → [(3,B)] → [(6,C)] → [(8,D)]

Walk backward:

(8) → (6) → stop at (3)

Insert after (3,B):

[(1,A)] → [(3,B)] → [(4,X)] → [(6,C)] → [(8,D)]

Time Complexity

- Worst case: scan entire list → **O(n)**

Prompt

Why is `min()` fast in this implementation?

Answer

Because the list is sorted:

- Minimum key is always at the **first position**

```
p = self.data.first()
```

O(1) time

Diagram

Front → [(2,B)] → [(5,A)] → [(7,D)]

↑

minimum

Prompt

How does `remove_min()` work?

Answer

1. Access the first position
2. Delete it from the list
3. Return its key-value pair

```
delete(self.data.first())
```

🕒 **O(1)** time

Diagram

Before:

```
[ (2,B) ] → [ (5,A) ] → [ (7,D) ]
```

After:

```
[ (5,A) ] → [ (7,D) ]
```

Prompt

What happens if `min()` or `remove_min()` is called on an empty queue?

Answer

An exception is raised:

```
Priority queue is empty
```

Time Complexity Summary

Operation	Time
<code>len()</code>	$O(1)$
<code>is_empty()</code>	$O(1)$
<code>add(k, v)</code>	$O(n)$
<code>min()</code>	$O(1)$
<code>remove_min()</code>	$O(1)$

📦 Space complexity: **O(n)**

Prompt

Compare sorted and unsorted list implementations of priority queues.

Answer

Operation	Unsorted List	Sorted List
add	$O(1)$	$O(n)$
min	$O(n)$	$O(1)$
remove_min	$O(n)$	$O(1)$

Space	$O(n)$	$O(n)$
-------	--------	--------

↻ Trade-off:

- **Unsorted list** → fast insert, slow delete
- **Sorted list** → slow insert, fast delete

Prompt

When is a sorted priority queue preferred?

Answer

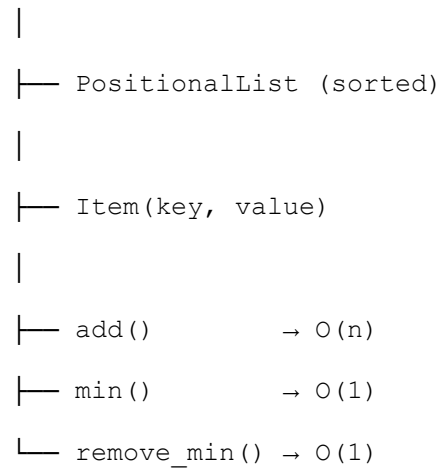
Use a sorted list when:

- Frequent `min()` / `remove_min()` operations
- Few insertions
- Dataset is small or moderately sized

✗ Not ideal for insertion-heavy workloads.

High-Level Structure Diagram

SortedPriorityQueue



Selection Sort (Using a Priority Queue)

Prompt

What is selection sort in the context of priority queues?

Answer

Selection sort is a sorting algorithm that:

1. Repeatedly **selects the smallest element**
2. Removes it
3. Appends it to the output in sorted order

When implemented using a **priority queue backed by an unsorted list**, the algorithm is naturally called **selection sort**, because the expensive step is repeatedly **selecting the minimum**.

Prompt

What are the two phases of priority-queue-based sorting?

Answer

◆ Phase 1 – Build the Priority Queue

- Insert all elements into priority queue P
- Using an **unsorted list**, each `add` is **$O(1)$**

◆ Phase 2 – Extract in Sorted Order

- Repeatedly call `remove_min()`
- Each removal requires scanning the list $\rightarrow$ **$O(\text{size of } P)$**

Prompt

Why is this algorithm called selection sort?

Answer

Because the **dominant operation** is:

Repeatedly **selecting the minimum element** from the unsorted collection

This is exactly the defining feature of classic selection sort.

Prompt

What is the running time of Phase 1?

Answer

- Insert n elements
- Each insertion = **$O(1)$**
- Total time:

$$O(n)$$

Prompt

Why is Phase 2 the bottleneck?

Answer

Each `remove_min()` scans the entire priority queue to find the minimum.

Removal	Size of P	Time
1st	n	$O(n)$
2nd	$n-1$	$O(n-1)$
3rd	$n-2$	$O(n-2)$
...	...	...
Last	1	$O(1)$

⌚ Total Time of Phase 2

$$O(n + (n-1) + (n-2) + \dots + 1)$$

Prompt

How do we simplify the total cost of Phase 2?

Answer

Using the known formula:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

Therefore:

$$O\left(\frac{n(n+1)}{2}\right) = O(n^2)$$

Prompt

What is the total running time of selection sort?

Answer

Phase	Time
Phase 1	$O(n)$
Phase 2	$O(n^2)$
Total	$O(n^2)$

✦ **Phase 2 dominates**, so selection sort is **quadratic time**.

Prompt

Show selection sort on $C = (7, 4, 8, 2, 5, 3)$

◆ Initial State

Collection C : $(7, 4, 8, 2, 5, 3)$

Priority Queue P : $()$

◆ Phase 1: Insert All Elements

Step (a): $C = (4, 8, 2, 5, 3)$ $P = (7)$

Step (b): $C = (8, 2, 5, 3)$ $P = (7, 4)$

Step (c): $C = (2, 5, 3)$ $P = (7, 4, 8)$

Step (d): $C = (5, 3)$ $P = (7, 4, 8, 2)$

Step (e): $C = (3)$ $P = (7, 4, 8, 2, 5)$

Step (f): $C = ()$ $P = (7, 4, 8, 2, 5, 3)$

⚠ Order inside P is **unsorted**

◆ Phase 2: Remove Minimum Repeatedly

Step	Removed Min	Sorted Output	Remaining P
(a)	2	(2)	(7,4,8,5,3)
(b)	3	(2,3)	(7,4,8,5)
(c)	4	(2,3,4)	(7,8,5)
(d)	5	(2,3,4,5)	(7,8)
(e)	7	(2,3,4,5,7)	(8)

(f)	8	(2,3,4,5,7,8)	()
-----	---	---------------	----

Visual Diagram (Selection Sort)

Unsorted P:

[7] [4] [8] [2] [5] [3]

↓ scan

min = 2 → output

[7] [4] [8] [5] [3]

↓ scan

min = 3 → output

[7] [4] [8] [5]

↓ scan

min = 4 → output

...

Each scan costs **linear time** ⌚

Prompt

Why is selection sort inefficient for large datasets?

Answer

- Always performs **n^2 comparisons**
- No early termination
- Does not adapt to partially sorted data

Insertion Sort (Using a Priority Queue)

Prompt

What is insertion sort when implemented using a priority queue?

Answer

Insertion sort is a sorting algorithm in which:

- Elements are **inserted one by one** into their **correct sorted position**

When the priority queue is implemented using a **sorted list**, the sorting process naturally becomes **insertion sort**, because:

- Insertion (`add`) is expensive
- Removal of minimum is cheap

Prompt

What are the two phases of insertion sort using a priority queue?

Answer

◆ Phase 1 – Insert All Elements (Sorted Order)

- Insert each element into a **sorted priority queue**
- Each insertion may require shifting elements

◆ Phase 2 – Remove Elements

- Repeatedly remove the minimum element
- Since the list is sorted, this is very fast

Prompt

Why does Phase 1 dominate the running time?

Answer

In a sorted list:

- To insert a new element, we may need to scan the list to find the correct position
- In the worst case, we scan the entire list

Insertion	Size of P	Time
1st	0	$O(1)$
2nd	1	$O(1)$
3rd	2	$O(2)$
...	...	...
nth	$n-1$	$O(n-1)$

Prompt

What is the worst-case time complexity of Phase 1?

Answer

$$O(1 + 2 + 3 + \dots + n)$$

Using the summation formula:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

So:

$$Phase\ 1 = O(n^2)$$

Prompt

Why is Phase 2 efficient in insertion sort?

Answer

- The list is already sorted
- `remove_min()` removes the **first element**
- Each removal takes **$O(1)$**

For n removals:

$$O(n)$$

Prompt

What is the total running time of insertion sort?

Answer

Phase	Time
Phase 1	$O(n^2)$
Phase 2	$O(n)$
Total	$O(n^2)$

★ Phase 1 dominates → **Worst case $O(n^2)$**

Prompt

Why does insertion sort have a best-case time of $O(n)$?

Answer

If the input is **already sorted**:

- Each new element is inserted at the end
- No scanning required
- Each insertion is $O(1)$

Thus:

$$\text{Best Case} = O(n)$$

★ Selection sort does **NOT** have this advantage

Prompt

Show insertion sort on $C = (7, 4, 8, 2, 5, 3)$

◆ Initial State

Collection C: (7, 4, 8, 2, 5, 3)

Priority Queue P: ()

◆ Phase 1: Sorted Insertions

- (a) Insert 7 $\rightarrow$ P = (7)
- (b) Insert 4 $\rightarrow$ P = (4, 7)
- (c) Insert 8 $\rightarrow$ P = (4, 7, 8)
- (d) Insert 2 $\rightarrow$ P = (2, 4, 7, 8)
- (e) Insert 5 $\rightarrow$ P = (2, 4, 5, 7, 8)
- (f) Insert 3 $\rightarrow$ P = (2, 3, 4, 5, 7, 8)

✦ Priority queue is **always sorted**

◆ Phase 2: Remove Minimum Repeatedly

- (a) Remove 2 $\rightarrow$ Output (2)
- (b) Remove 3 $\rightarrow$ Output (2, 3)
- (c) Remove 4 $\rightarrow$ Output (2, 3, 4)
- (d) Remove 5 $\rightarrow$ Output (2, 3, 4, 5)
- (e) Remove 7 $\rightarrow$ Output (2, 3, 4, 5, 7)
- (f) Remove 8 $\rightarrow$ Output (2, 3, 4, 5, 7, 8)

Visual Diagram (Insertion Sort)

Insert 7:

[7]

Insert 4:

[4][7]

Insert 8:

[4][7][8]

Insert 2:

[2][4][7][8]

Insert 5:

[2][4][5][7][8]

Insert 3:

[2] [3] [4] [5] [7] [8]

☛ Each insertion maintains sorted order

Comparison: Selection vs Insertion Sort

Feature	Selection Sort	Insertion Sort
PQ Type	Unsorted List	Sorted List
Insert	$O(1)$	$O(n)$
Remove Min	$O(n)$	$O(1)$
Worst Case	$O(n^2)$	$O(n^2)$
Best Case	$O(n^2)$	$O(n)$